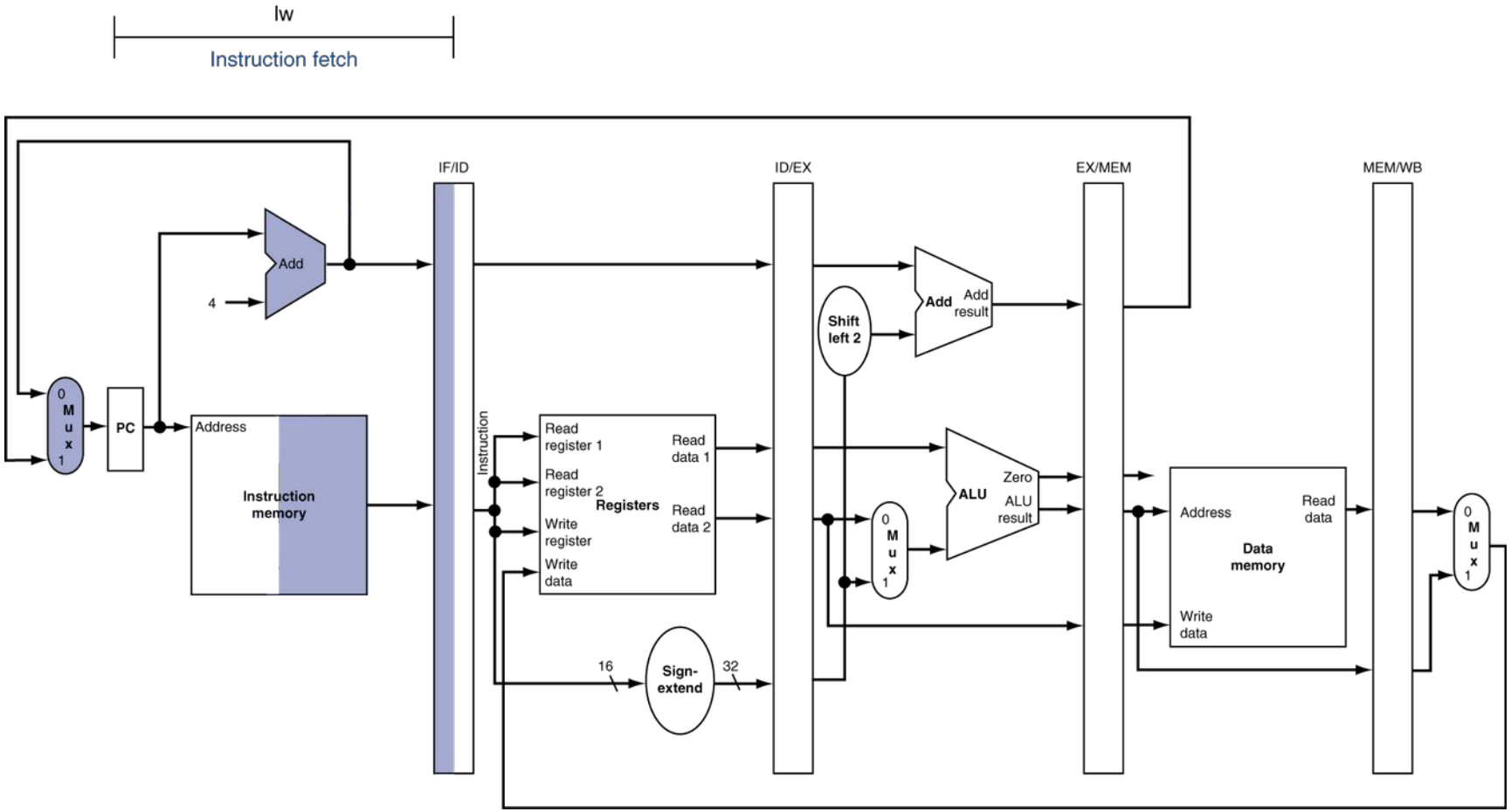
Chapter 4

Pipelining

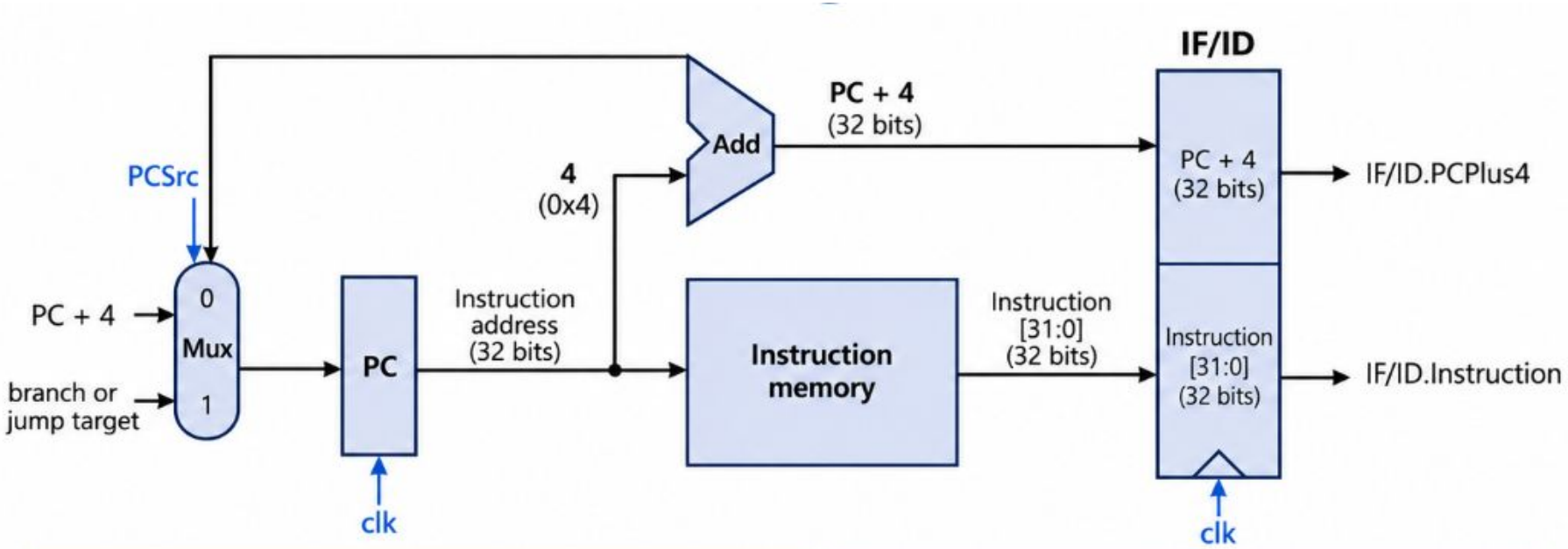
Pipeline Operation

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - “Single-clock-cycle” pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. “multi-clock-cycle” diagram
    - Graph of operation over time
- We’ll look at “single-clock-cycle” diagrams for load & store

IF for Load, Store, …



Instruction fetch (Cont.)



Key operations (common for all instructions)

1. **Select next PC**
   - PCSrc = 0: PC + 4 (sequential execution)
   - PCSrc = 1: branch or jump target
2. **Fetch instruction**
   - Use PC to read the 32-bit instruction from instruction memory
3. **Compute PC + 4**
   - Adder adds constant 4 to the current PC
4. **Store in IF/ID register**
   - Save PC + 4 and the fetched instruction for use in the next stage (ID)

Example (sequential execution)

Assume PC = 0x00400020 and PCSrc = 0.

PC (to instruction memory)	0x00400020
PC + 4	0x00400024
Next PC	0x00400024

Fetched instruction (32 bits)

```
0x8C890004      # lw $t1, 4($a0)
```

Instruction fetch (Cont.)



Q1. A lw instruction has just completed the IF stage. Which information must be stored in the IF/ID pipeline register for the next stage?

A. ALU result and destination register number

B. Instruction and PC + 4

C. Memory read data and RegWrite

D. Register operands and sign-extended immediate

A1.

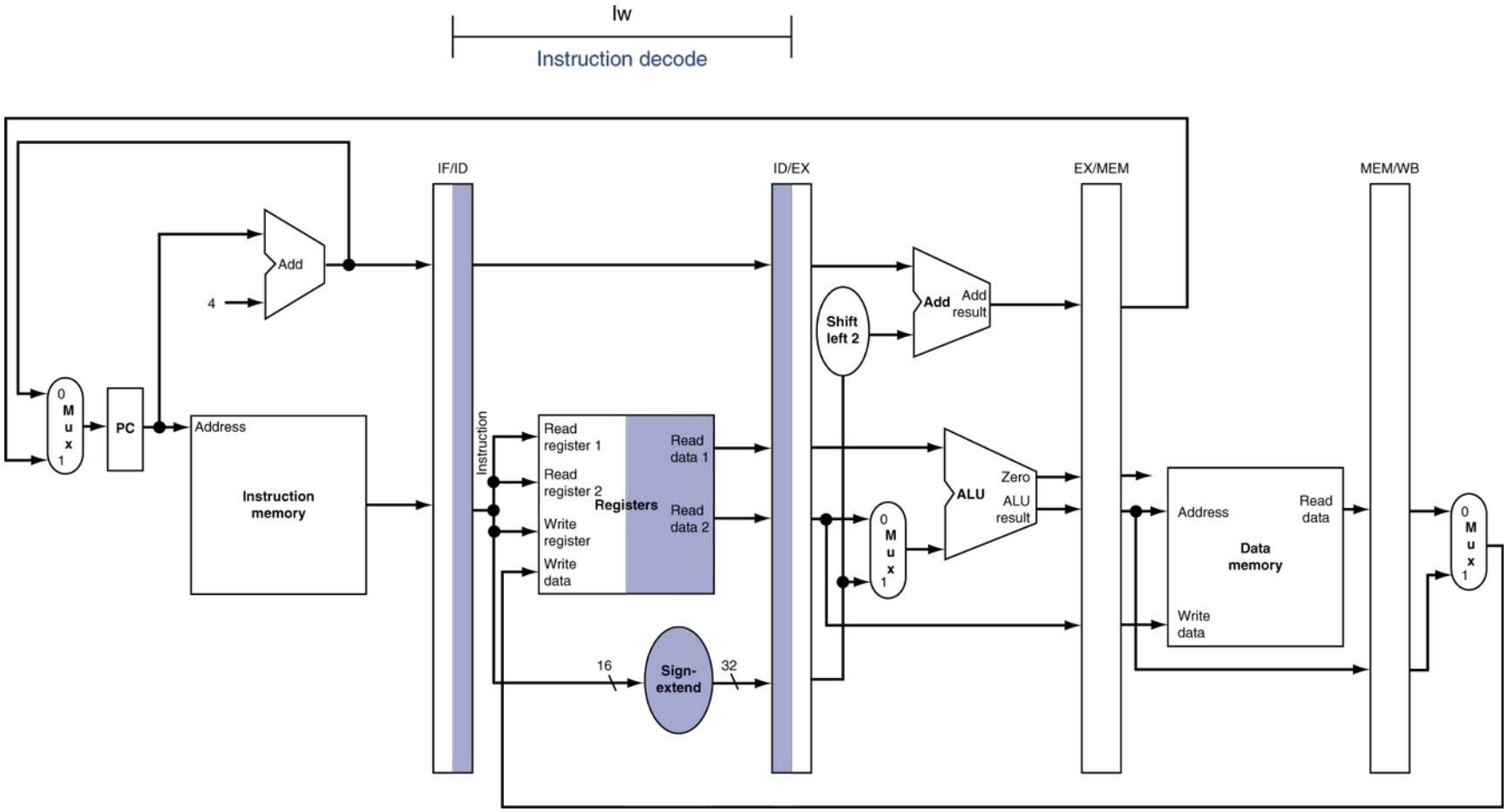
B. Instruction and PC + 4

A. Needed for decoding and register selection in ID.

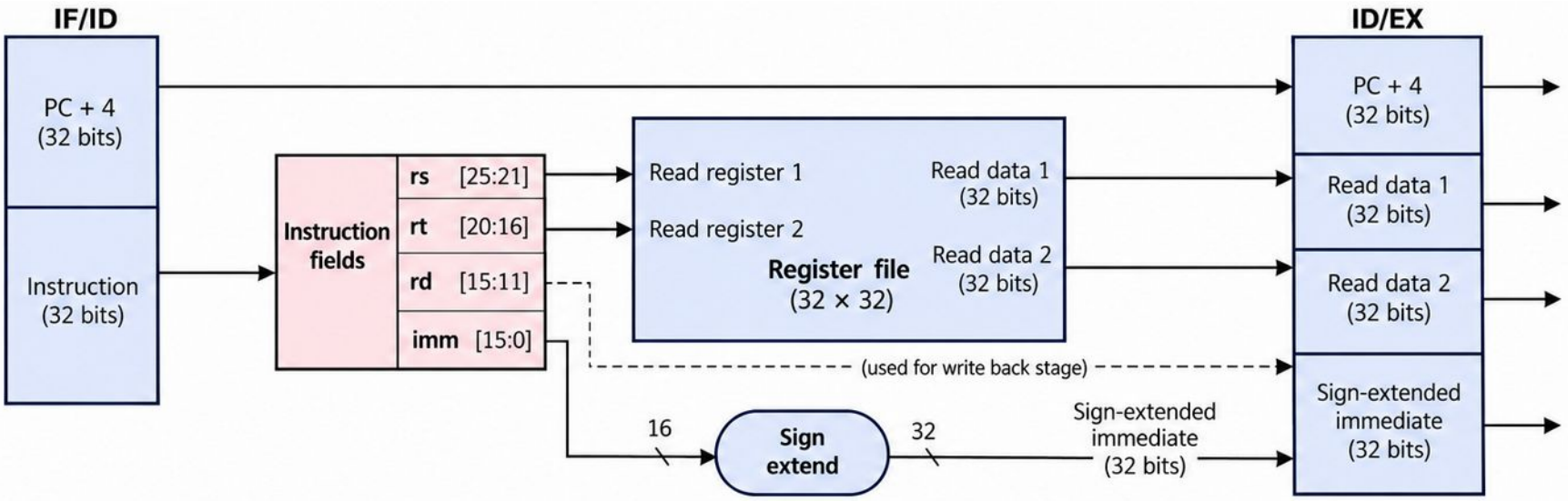
B. PC + 4 is preserved for instructions in later stages.

C. Register operands and sign-extended immediate are generated during ID.

ID for Load, Store, …



Instruction Decode (Cont.)



Operation	Load (lw)	Store (sw)	R-type (add, sub, ...)
Read rs (base/source 1)	✓	✓	✓
Read rt (source 2 / data)	–	✓ (store data)	✓ (source 2)
Read rd	–	–	– (used in WB stage)
Immediate [15:0] (sign-extended)	✓ (offset)	✓ (offset)	–
Outputs to ID/EX	PC + 4 Read data 1 (rs) Sign-extended imm	PC + 4 Read data 1 (rs) Read data 2 (rt) Sign-extended imm	PC + 4 Read data 1 (rs) Read data 2 (rt)

Example: lw $t1, 4($a0)

Instruction (32 bits)

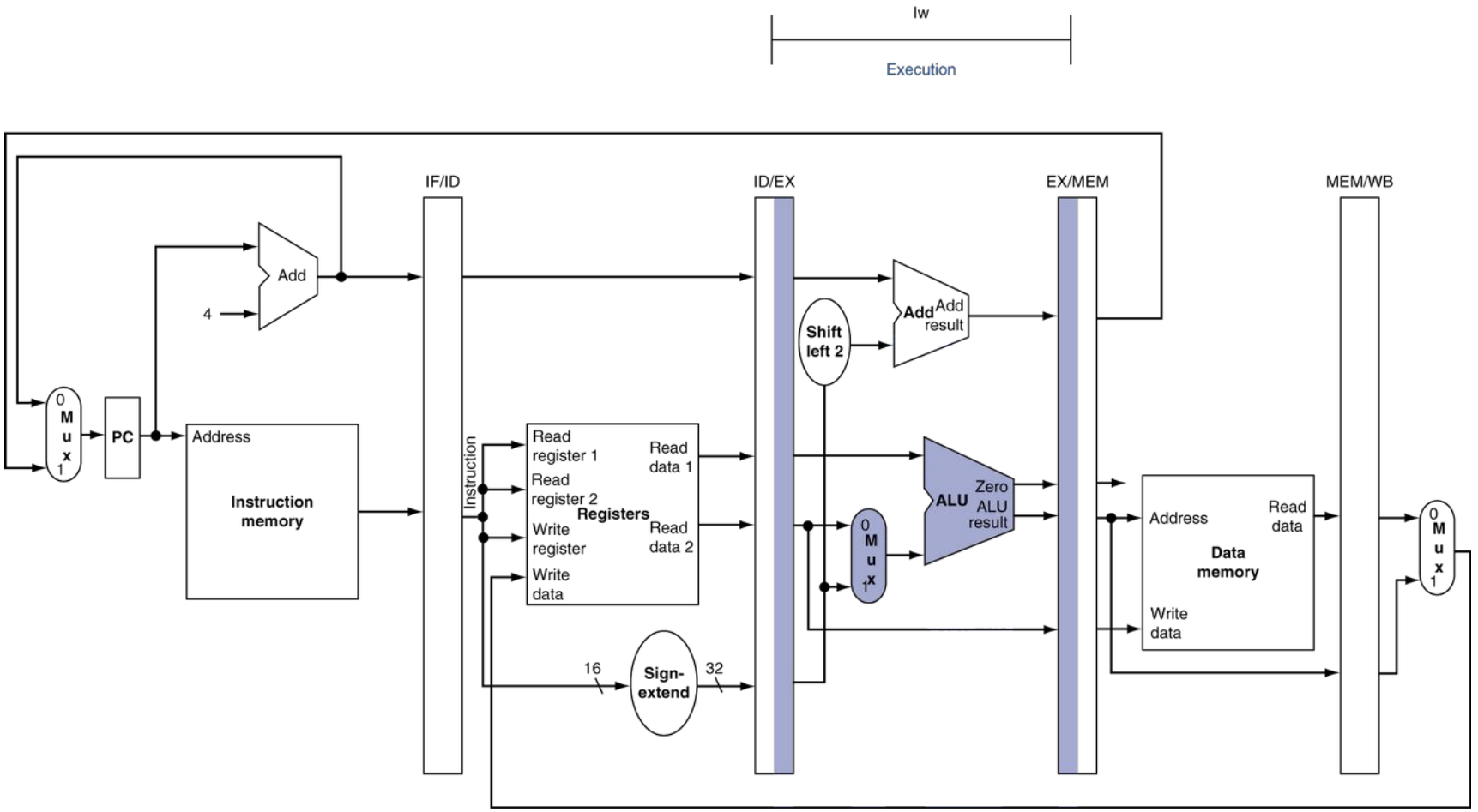
0x8C890004

31–26	25–21	20–16	15–0
opcode (35) 100011	rs ($a0) 01000	rt ($t1) 01001	immediate (4) 0000 0000 0000 0100

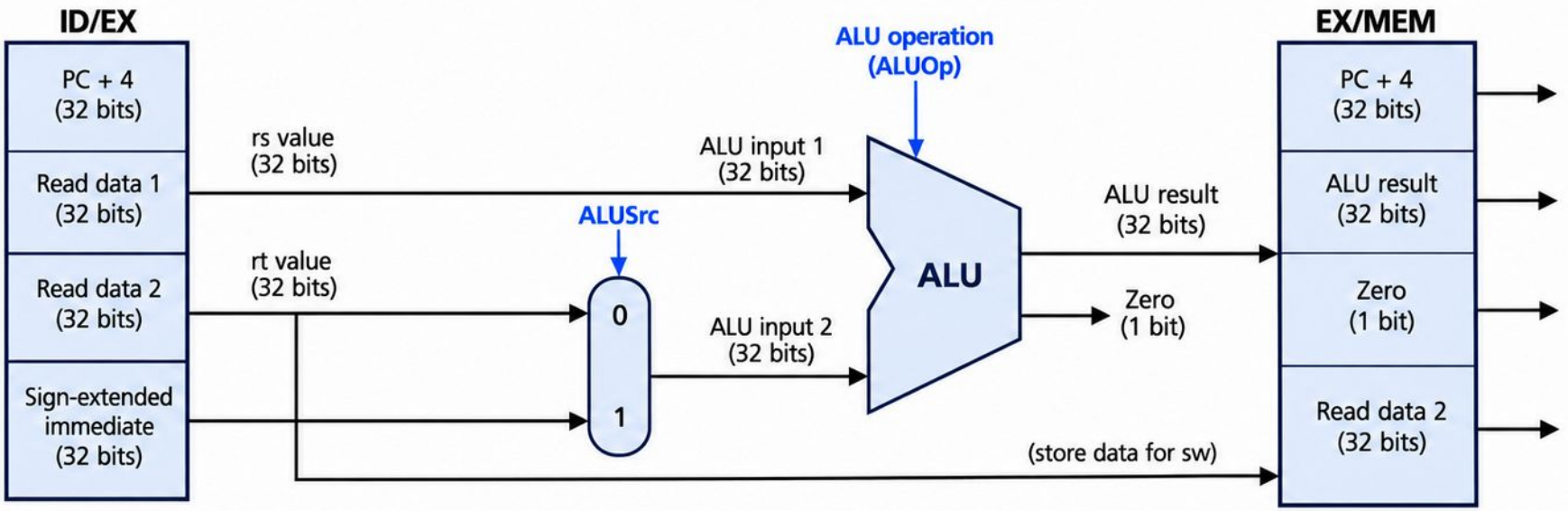
ID stage results

Read data 1 (rs = $a0)	value in $a0 (base address)
Read data 2 (rt = $t1)	not used (–)
Sign-extended immediate	0x00000004
PC + 4	PC + 4 (sequential address)
Pass to ID/EX	PC + 4, read data 1, sign-extended immediate

EX for Load


lw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32

Execute (Cont.)



Operation	Load (lw)	Store (sw)	R-type (add, sub, ...)
ALU operation	Add (address calc.)	Add (address calc.)	According to funct (add, sub, and, ...)
ALU input 1	rs value (base address)	rs value (base address)	rs value
ALU input 2 (ALUSrc)	sign-extended imm (offset) (ALUSrc = 1)	sign-extended imm (offset) (ALUSrc = 1)	rt value (ALUSrc = 0)
ALU result	effective address	effective address	arithmetic/logic result
Zero output	not used	not used	set if result = 0 (used by branches)
Forward to EX/MEM	ALU result (effective address)	ALU result (effective address), Read data 2 (store data)	ALU result

Example: lw $t1, 4($a0)

Inputs from ID/EX

Read data 1 (rs = $a0)	0x10010000
Read data 2 (rt = $t1)	not used
Sign-extended immediate	0x00000004

ALU operation in EX stage

ALU result = 0x10010000 + 0x00000004 = **0x10010004**

Outputs to EX/MEM

ALU result	0x10010004 (effective address)
Zero	not used (–)
Read data 2	value of $t1 (forwarded for sw)

Execute (Cont.)



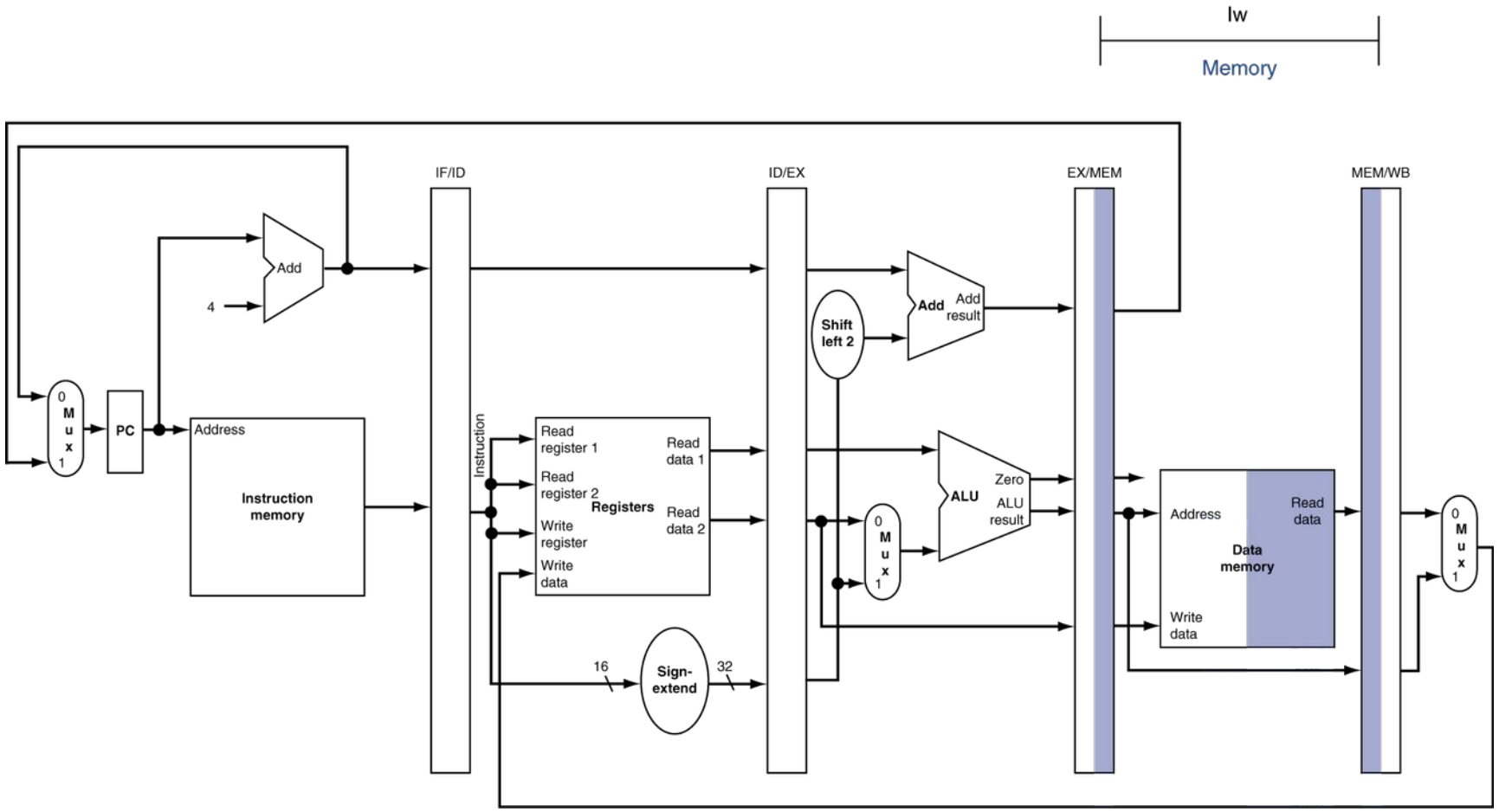
Q2. Consider the following two instructions:
lw $t0, 12($s1)
add $t2, $s1, $t3
Assume $s1 = 100 and $t3 = 25. For each instruction, determine the two ALU inputs, the value of ALUSrc, and the ALU result during the EX stage.

A2.

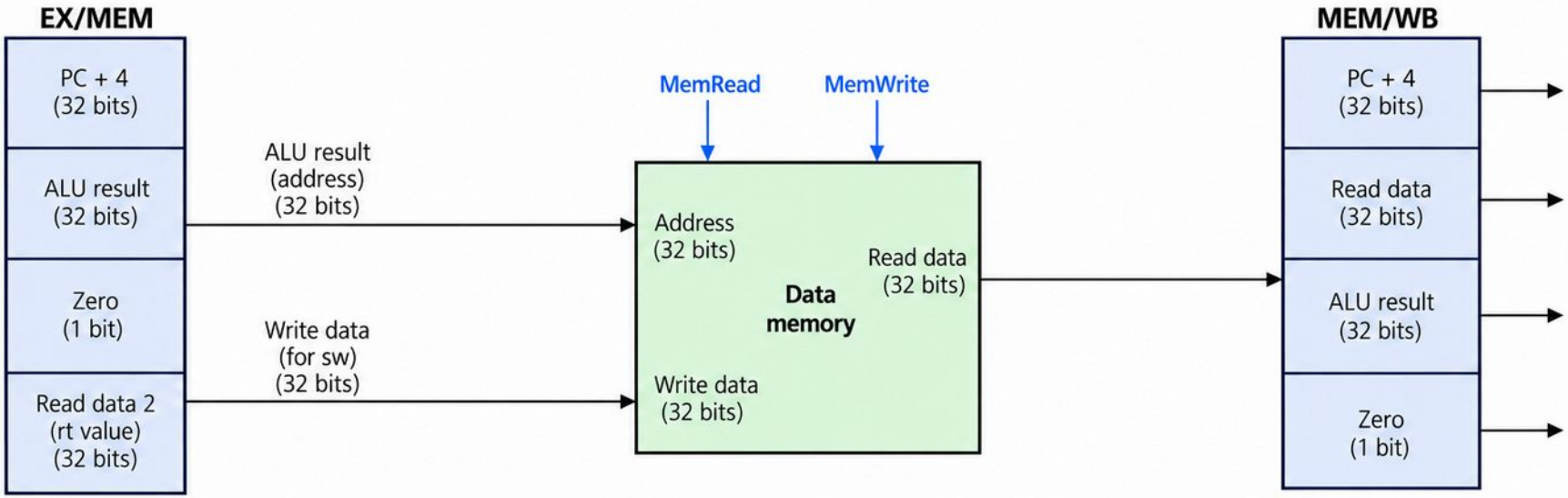
Instruction	ALU Input 1	ALU Input 2	ALUSrc	ALU Result
lw	100	12	1	112
add	100	25	0	125

- **lw uses the base register and sign-extended immediate to calculate the effective address.**
- **add uses two register operands to perform arithmetic.**

MEM for Load


lw
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
16
Sign-extend
32

Memory Access (Cont.)



Operation	Load (lw)	Store (sw)	R-type (add, sub, ...)
Memory access	Read from Data memory	Write to Data memory	No memory access
Address input	ALU result (effective address)	ALU result (effective address)	ALU result (usually not used)
Write data	—	rt value (from EX/MEM)	—
Read data	value from Data memory	—	—
Control signal	MemRead = 1 MemWrite = 0	MemRead = 0 MemWrite = 1	MemRead = 0 MemWrite = 0
Outputs to MEM/WB	Read data (from memory)	ALU result (effective address)	ALU result (ALU computation)
Zero output	not used	not used	not used

Example: lw $t1, 4($a0)

Inputs from EX/MEM

ALU result (address)	0x10010004
Write data (rt = $t1)	not used (—)
MemRead	1
MemWrite	0

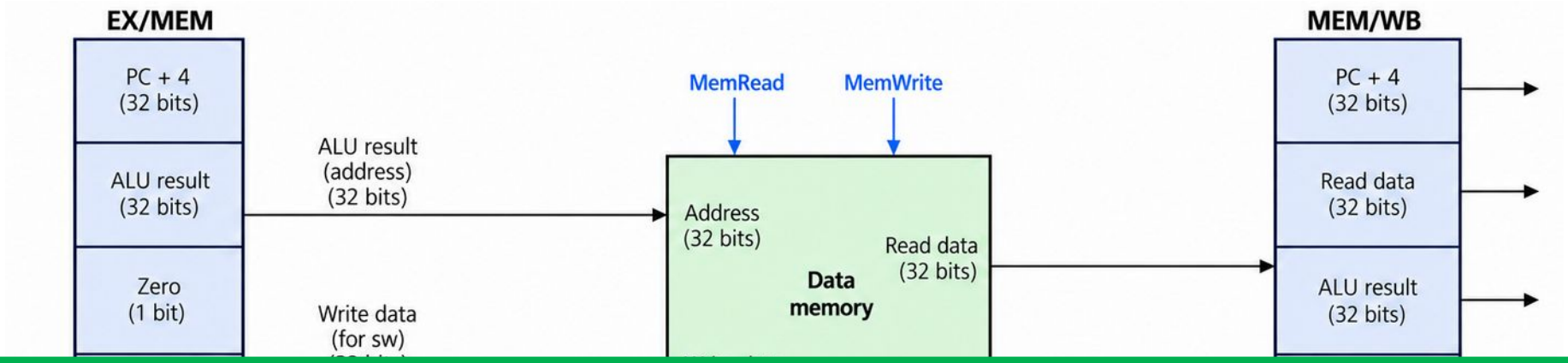
Memory access in MEM stage

Data memory read:
Read data = Mem[0x10010004] = **0xDEADBEEF**

Outputs to MEM/WB

PC + 4	PC + 4 (sequential address)
Read data	0xDEADBEEF (value loaded from memory)
ALU result	0x10010004 (effective address)
Zero	not used (—)

Memory Access (Cont.)



Q3. Which of the following are stored in the MEM/WB pipeline register? (Select all that apply.)

A.ALU Result

B. Memory Read Data

C. Sign-extended Immediate

D. Destination Register Number

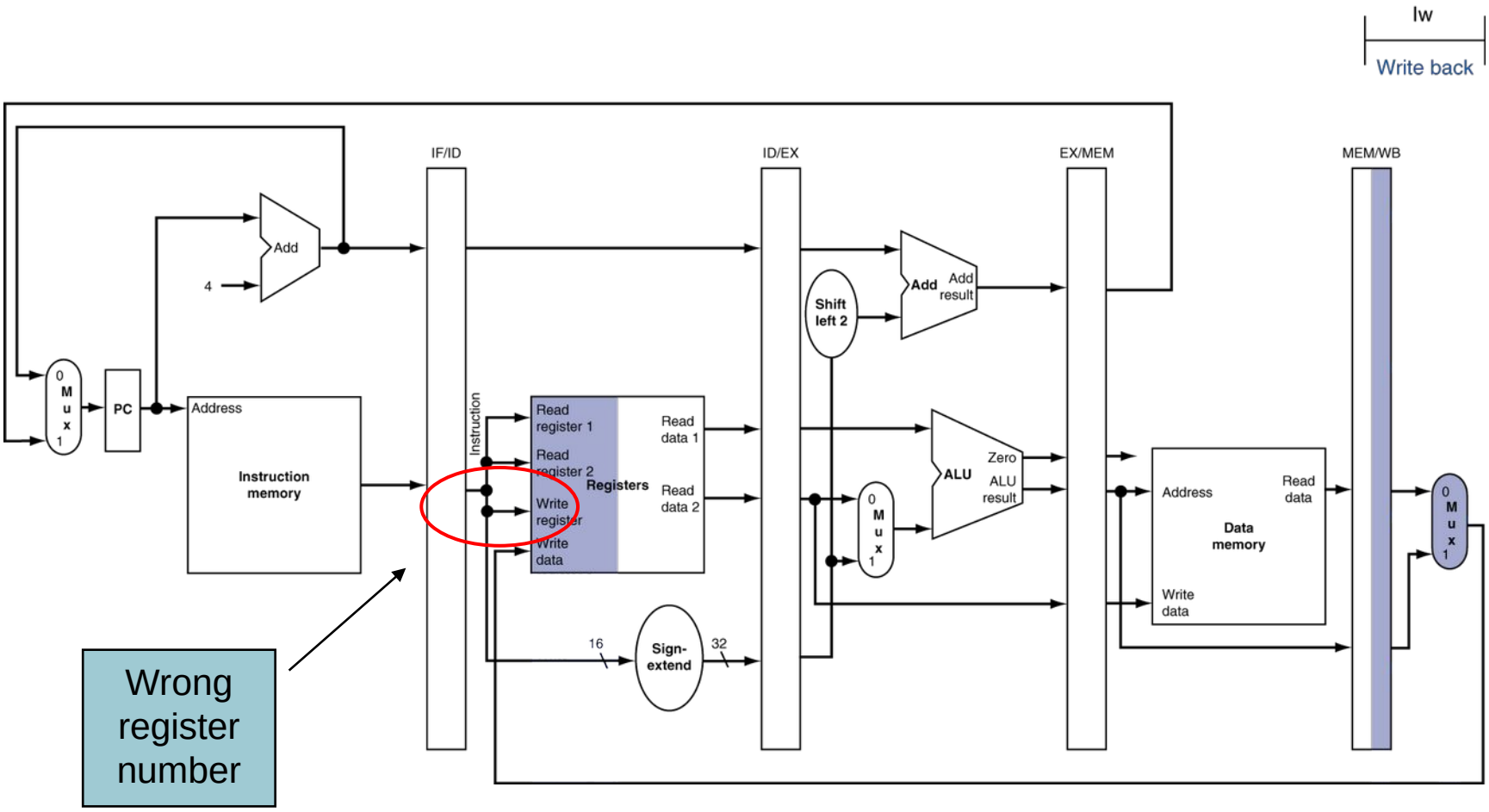
E. MemtoReg

F. ALUSrcG. RegWrite

A3.

A, B, D, E, G

WB for Load


lw
Write back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32
Wrong register number

Wrong Register Number

If the WB stage uses the register number from the current ID stage, it may write to the wrong register.

Instruction	Cycle	1	2	3	4	5	6	7
I_1: **lw** $t1, 4($a0)		IF	ID	EX	MEM	WB		
I_2: **add** $t2, $t3, $t4			IF	ID	EX	MEM	WB	
I_3: **sub** $t5, $t6, $t7				IF	ID	EX	MEM	WB
I_4: **and** $t8, $t9, $t10					IF	ID	EX	MEM

Cycle 5:
The load instruction (I_1) is in WB stage and should write to **$t1 (9)**.

In the same cycle, the current ID stage instruction is I_4 (and), whose destination register number is **$t8 (24)**.

1. **Multiple instructions in pipeline**
   In cycle 5, I_1 (lw) is in WB stage, and I_4 (and) is in ID stage.
2. **Using current ID stage register number**
   If the register file uses the register number from the current ID stage (I_4: **$t8**), the WB stage will write to **$t8** instead of **$t1**.
3. **Wrong register is written**
   The load value (from memory) will be written to **$t8** instead of **$t1**.

Conclusion

The **destination register number** (rt or rd) for each instruction must be carried with that instruction through the pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) to ensure the WB stage writes to the **correct register**.

Wrong Register Number

Q4. Consider the following instructions in a five-stage pipeline without stalls

I1: lw $t1, 4($s0)

I2: add $t2, $s1, $s2

I3: sub $t3, $s3, $s4

I4: and $t4, $s5, $s6

When I1 reaches the WB stage, I4 is in the ID stage. If the destination register number is incorrectly taken from the current ID stage, which register will be written instead of $t1?

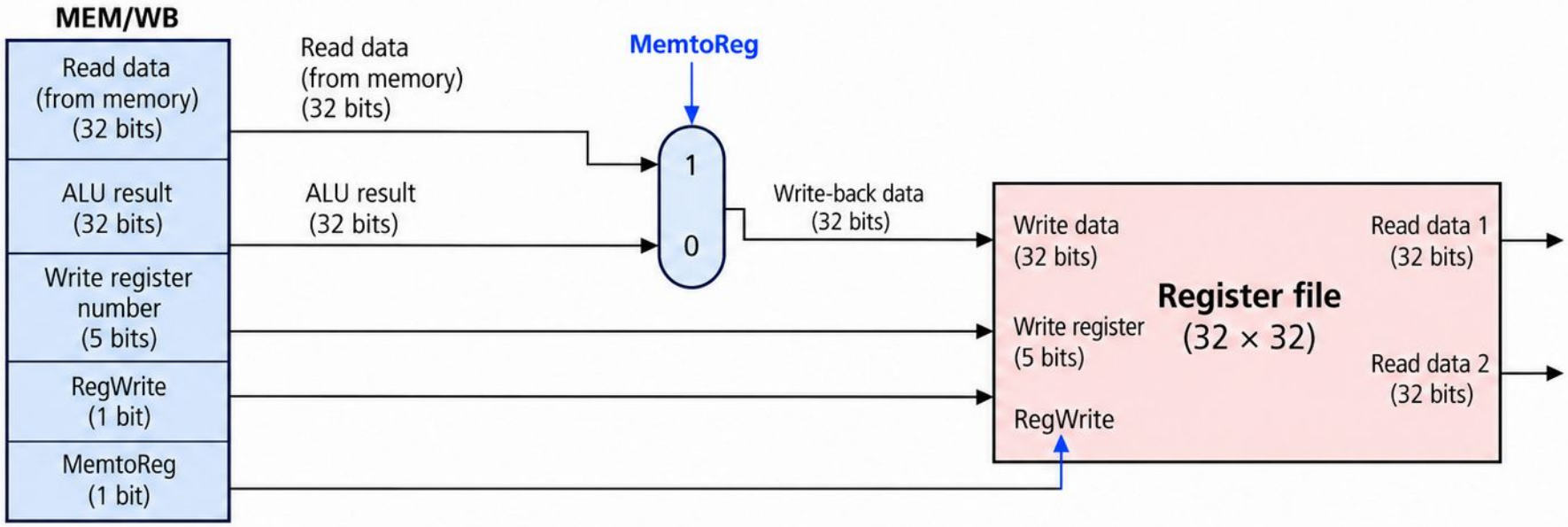
A4.

$t4

The WB stage would incorrectly use the destination register number of I4 ($t4). The correct destination register number ($t1) must travel with I1 through the pipeline registers.

The load value (from memory) will be written to $t8 instead of $t1.

Write Back (Cont.)



Operation	Load (lw)	Store (sw)	R-type (add, sub, ...)
Data written to register file	Data from memory	None	ALU result
MemtoReg	1	– (don't care)	0
RegWrite	1	0	1
Write register	rt (from MEM/WB)	–	rd (from MEM/WB)
Write-back data	Read data (from memory)	–	ALU result
Inputs from MEM/WB	Read data ALU result Write register MemtoReg, RegWrite	(not used for write) Write register MemtoReg, RegWrite	Read data ALU result Write register MemtoReg, RegWrite

Example: lw $t1, 4($a0)

Inputs from MEM/WB

Read data (from memory)	0xDEADBEEF
ALU result	0x10010004
Write register number (rt = $t1)	9 (5 bits)
MemtoReg	1
RegWrite	1

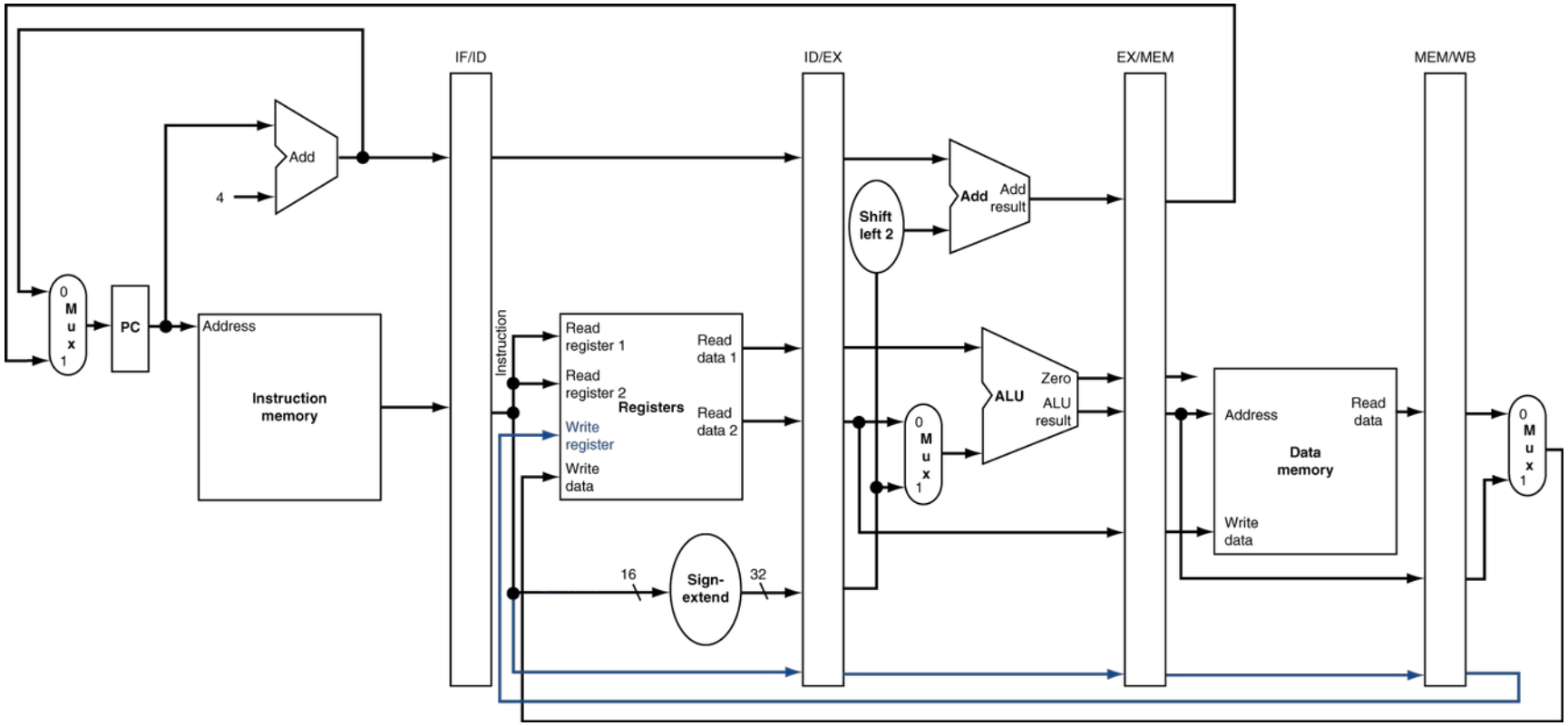
Write-back operation

Write register $t1 ← **0xDEADBEEF**

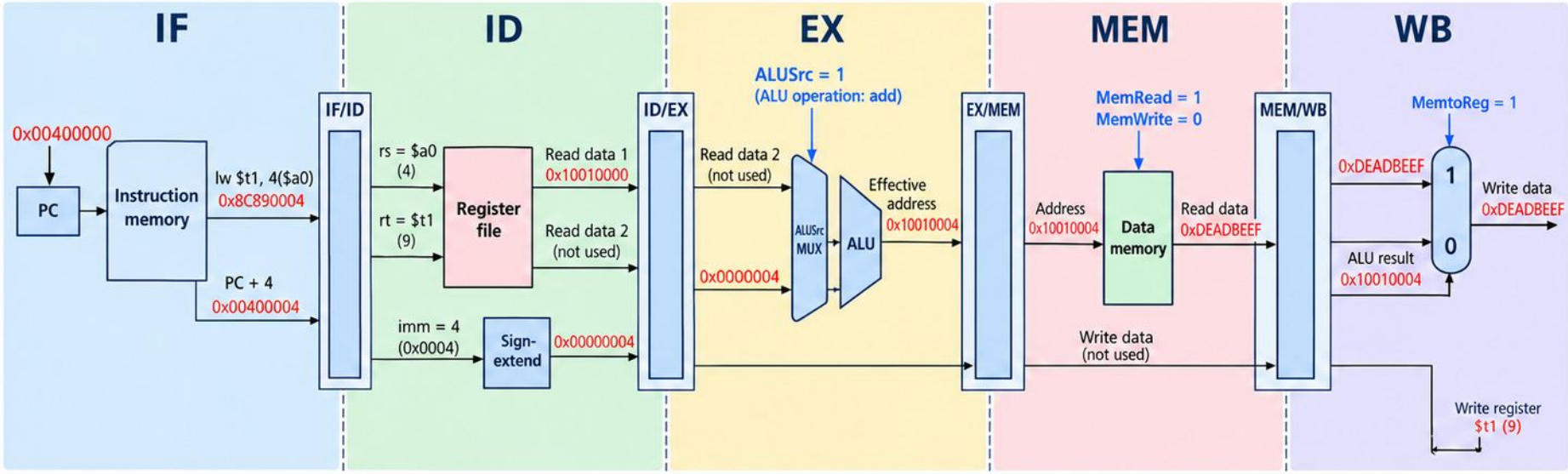
Result in register file

Register	$t1 (register number 9)
Written value	0xDEADBEEF

Corrected Datapath for Load



Complete Load Inst. Tracing



IF — Instruction Fetch

Instruction:
lw $t1, 4($a0)

PC:
0x00400000

Instruction immory →
0x8C890004

PC + 4:
0x00400004

Outputs to IF/ID:
- Instruction = lw $t1, 4($a0)
- PC + 4 = 0x00400004

ID — Decode & Register Read

Register file read:
rs = $a0 → 0x10010000
rt = $t1 → (not used)

Sign-extend immediate:
4 → 0x00000004

Outputs to ID/EX:
- Read data 1 = 0x10010000
- Read data 2 = (not used)
- Imm = 0x00000004
- rs = 4, rt = 9
- Control signals

EX — Address Calculation

ALUSrc = 1 → use immediate

ALU operation: add

0x10010000 + 0x00000004
= 0x10010004

Outputs to EX/MEM:
- ALU result = 0x10010004
- Read data 2 = (not used)
- rt = 9 (destination register)
- Control signals

MEM — Memory Read

MemRead = 1, MemWrite = 0

Address:
0x10010004

Data memory read:
0xDEADBEEF

Outputs to MEM/WB:
- Read data = 0xDEADBEEF
- ALU result = 0x10010004
- rt = 9
- Control signals

WB — Write Back

MemtoReg = 1

Write register = $t1 (9)

Write data = 0xDEADBEEF

Result:
$t1 = 0xDADBEEF

Q5. Consider the following instruction: lw $t2, 8($s0)
Assume $s0 = 0x10010000 and Memory[0x10010008] = 0x0000002A. Complete the pipeline tracing table.

Stage	Operation / Result
IF	Fetch the instruction
ID	Read $s0 = ?
EX	Effective Address = ?
MEM	Memory Read Data = ?
WB	Register Updated = ?

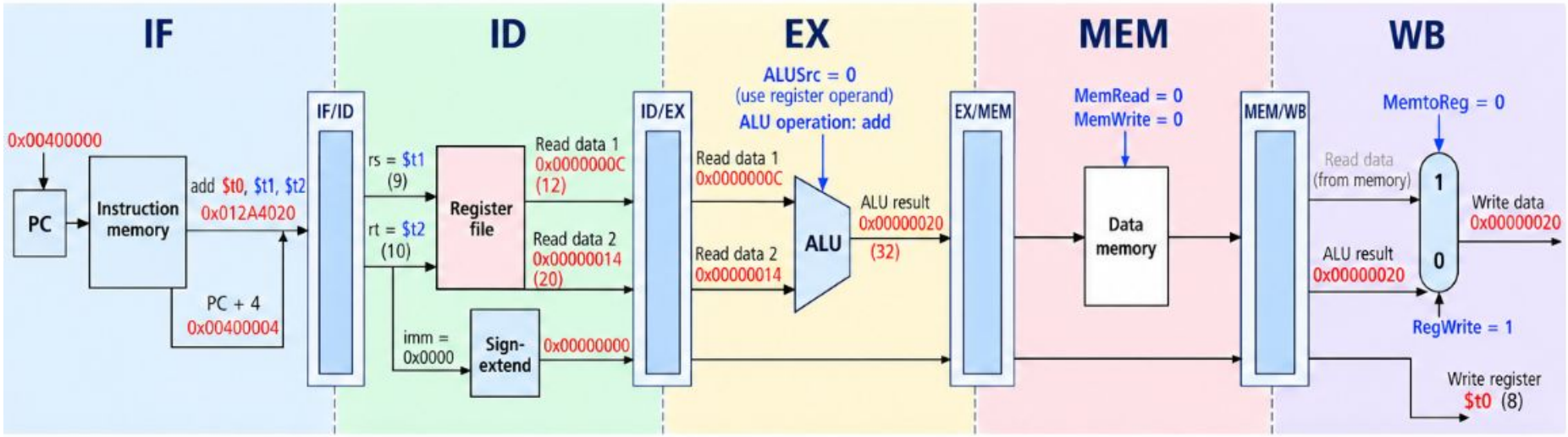
A5.

Stage	Operation / Result
IF	Fetch lw $t2, 8($s0)
ID	0x10010000
EX	0x10010008
MEM	0x0000002A
WB	$t2 = 0x0000002A

Complete R-format Instruction

Instruction: add $t0, $t1, $t2

Assume $t1 = 0x0000000C (12), $t2 = 0x00000014 (20).



IF — Instruction Fetch

Instruction:
add $t0, $t1, $t2

PC:
0x00400000

Instruction memory →
0x012A4020

PC + 4:
0x00400004

Outputs to IF/ID:
- Instruction = add $t0, $t1, $t2 (0x012A4020)
- PC + 4 = 0x00400004

ID — Decode & Register Read

Register file read:
rs = $t1 (9) → 0x0000000C (12)
rt = $t2 (10) → 0x00000014 (20)

Sign-extend immediate:
0x0000 → 0x00000000

Outputs to ID/EX:
- Read data 1 = 0x0000000C
- Read data 2 = 0x00000014
- Imm = 0x00000000
- rs = 9, rt = 10, rd = 8 ($t0)
- Control signals

EX — Execute

ALUSrc = 0 (use register operand)

ALU operation: add

ALU calculation:
0x0000000C + 0x00000014
= 0x00000020 (32)

Outputs to EX/MEM:
- ALU result = 0x00000020
- Read data 2 = 0x00000014 (not used for R-type)
- rd = 8 ($t0)
- Control signals

MEM — Memory Access

MemRead = 0, MemWrite = 0

No memory operation
(for R-type)

Outputs to MEM/WB:
- Read data = (not used)
- ALU result = 0x00000020
- rd = 8 ($t0)
- Control signals

WB — Write Back

MemtoReg = 0 (select ALU result)

Write register = $t0 (8)

Write data = 0x00000020 (32)

Result:
$t0 = 0x00000020 (32)

Section 4-2 Quick Questions

Q1. What information must be stored in the IF/ID pipeline register?

Q2. For lw and add, determine the ALU inputs, ALUSrc, and ALU result.

Q3. Which data and control signals are stored in the MEM/WB pipel ine register?

Q4. If WB incorrectly uses the destination register number from ID, which register will be written?

Q5. Trace a lw instruction through all five pipeline stages and deter mine the results.

Exercise 1 — Pipeline Register Consistency

Consider three instructions in a five-stage pipeline:

```
I1: lw   $t0, 8($s0)
I2: add  $t1, $s1, $s2
I3: sub  $t2, $s3, $s4
```

ID/EX	EX/MEM	MEM/WB
I3	I2	I1

EX/MEM contains ALU Result = 45, but its destination register number is $t2. Identify the incorrect field and provide the correct value.

Answer

Incorrect field: EX/MEM destination register number.

Correct value: $t1 (the destination register of I2).

Key Point: Data and register numbers must belong to the same instruction.

Exercise 2 — Pipeline Stage Timing

Consider a five-stage pipeline without stalls or forwarding:

```
I1: add  $t0, $s0, $s1
I2: sub  $t2, $t0, $s2
I3: and  $t3, $s3, $s4
```

When I2 is in the ID stage, identify the stages occupied by I1 and I3. Has I1 already written its result to $t0?

Explain your answer based on the five-stage pipeline sequence.

Answer

I1: EX; I2: ID; I3: IF.

No. I1 writes $t0 during WB, which occurs later.

Key Point: Instructions overlap in different stages; WB occurs after EX and MEM.

Exercise 3 — Reverse Pipeline Reasoning

The following values are observed for one instruction:

EX: ALU Result = 0x10010014
MEM: Memory Read Data = 0x00000064
WB: $t5 receives 0x00000064

Assume $s0 = 0x10010000. Determine the instruction type, destination register, and immediate offset.

Write the complete MIPS instruction.

Answer

Instruction: lw $t5, 20($s0)

Offset = 0x14 = 20; destination register = $t5.

Key Point: EX, MEM, and WB results can reveal the original load instruction.